PrintOnline.et v3.2 — Chapa Integration E2E Testing Plan

> **Version:** 3.2.0
> **Date:** 2026-03-17
> **Status:** READY FOR EXECUTION
> **Testing Mode:** Chapa Sandbox/Test Environment

---

1. Prerequisites & Setup

1.1 Environment Variables Required

```
# .env.local
CHAPA_SECRET_KEY=CHASECK_TEST-xxxxxxxxxxxxxxxxxxxxxxx
CHAPA_BASE_URL=https://api.chapa.co/v1
NEXT_PUBLIC_APP_URL=http://localhost:3000
```

1.2 Chapa Test Credentials

Card Type	Card Number	Expiry	CVV	PIN	OTP
Successful Payment	4242 4242 4242 4242	Any future date	123	1234	123456
Failed Payment	4000 0000 0000 0002	Any future date	123	1234	123456
Insufficient Funds	4000 0000 0000 9995	Any future date	123	1234	123456

1.3 Local Development Setup

```
# Terminal 1: Start Next.js dev server
pnpm dev

# Terminal 2: Start ngrok tunnel for webhook testing
ngrok http 3000
```

Note: Update `NEXT_PUBLIC_APP_URL` with ngrok URL when testing webhooks (e.g., `https://abc123.ngrok.io`)

---

2. Security Requirements Checklist

2.1 Server-Side Security

Requirement	Implementation	Test Method

Requirement	Implementation	Test Method
Secret Key Protection	`CHAPA_SECRET_KEY` only used server-side in `lib/chapa.ts`	Verify no exposure in browser Network tab
API Route Protection	`app/api/orders/route.ts` requires authenticated session	Test unauthenticated request returns 401
Idempotent Verification	`app/api/payments/verify/route.ts` checks existing payment status	Call verify twice, second should return cached result
Webhook Idempotency	`app/api/payments/webhook/route.ts` prevents double processing	Send duplicate webhook, should not update twice
Input Validation	Zod schemas validate all inputs	Send malformed data, should return 400
Error Handling	Try-catch blocks with proper error responses	Trigger errors, verify no stack traces exposed

2.2 Client-Side Security

Requirement	Implementation	Test Method
No Secret Exposure	Client never accesses `CHAPA_SECRET_KEY`	Search bundle for secret key
Redirect-Based Payment	User redirected to Chapa hosted page	Verify no card inputs on our domain
HTTPS Enforcement	Production uses HTTPS	Test mixed content warnings
CSRF Protection	Next.js built-in CSRF protection	Verify token validation

3. Test Scenarios

3.1 Happy Path — Successful Payment Flow

Test ID: `E2E-001`
Priority: P0 — Critical

Step	Action	Expected Result
1	Add items to cart	Cart updates with items
2	Navigate to `/cart`	Cart page displays items and total
3	Click "Proceed to Checkout"	Redirects to `/order-summary?step=1`
4	Fill delivery information (Step 1)	Form validates, "Continue" enabled
5	Click "Continue"	Advances to Step 2 (Review)
6	Review order details (Step 2)	All items, prices, delivery info correct

Step	Action	Expected Result
7	Accept terms and conditions	Checkbox checked
8	Click "Continue to Payment"	Advances to Step 3 (Payment)
9	Verify payment step displays	Shows "Chapa (Hosted)" method, correct total
10	Click "Pay & Place Order"	Button shows "Redirecting to Chapa..."
11	Redirected to Chapa hosted page	Chapa payment form loads
12	Enter test card: `4242 4242 4242 4242`	Card accepted
13	Complete payment (PIN: 1234, OTP: 123456)	Payment processes
14	Redirected to `/order-confirmation?tx_ref=...`	Page loads with spinner
15	Verification completes	Shows "Payment Received" success state
16	Verify order details displayed	Order number, items, total correct
17	Check email received	Confirmation email sent
18	Verify cart cleared	Cart empty on refresh
19	Check database	Order status: `order_confirmed`, payment_status: `paid`

Success Criteria:

- ☑ Order created with `pending_payment` status
- ☑ Chapa `checkout_url` received and used for redirect
- ☑ Payment verified via `/api/payments/verify`
- ☑ Order status updated to `order_confirmed`
- ☑ `payment_completed_at` timestamp recorded
- ☑ Confirmation email sent after verification

3.2 Failed Payment — Card Declined

Test ID: `E2E-002`
Priority: P0 — Critical

Step	Action	Expected Result
1-11	Follow happy path steps 1-11	Same as above
12	Enter declined card: `4000 0000 0000 0002`	Card entered
13	Attempt payment	Payment fails
14	User returns to site (via return_url or manual)	Arrives at order-confirmation
15	Verification runs	Returns `success: false`

Step	Action	Expected Result
16	Error UI displayed	Shows "Payment Verification Failed" message
17	Retry options available	"Try Again" and "Contact Support" buttons

Success Criteria:

- ☑ Order remains in `pending_payment` status
- ☑ No confirmation email sent
- ☑ Cart NOT cleared (user can retry)
- ☑ Clear error messaging displayed

3.3 User Cancels Payment on Chapa

Test ID: `E2E-003`
Priority: P1 — High

Step	Action	Expected Result
1-11	Follow happy path steps 1-11	Same as above
12	On Chapa page, click "Cancel" or close tab	User abandons payment
13	User manually navigates to `/order-confirmation?tx_ref=...`	Page loads
14	Verification runs	Chapa returns failed/canceled status
15	Error UI displayed	Shows appropriate failure message

Success Criteria:

- ☑ Order remains in `pending_payment` status
- ☑ No status update to `order_confirmed`
- ☑ User can retry payment (future enhancement)

3.4 Network Failure During Verification

Test ID: `E2E-004`
Priority: P1 — High

Step	Action	Expected Result
1	Complete successful payment on Chapa	Payment succeeds
2	Simulate network failure during verification	Disconnect network briefly
3	Verification API call fails	Error caught gracefully

Step	Action	Expected Result
4	User sees error state	"Verification Failed" with retry option
5	User clicks "Retry Verification"	Verification re-attempts
6	Network restored, verification succeeds	Order confirmed

Success Criteria:

- ☑ Graceful error handling (no white screen)
- ☑ Retry mechanism available
- ☑ Idempotent verification prevents double-processing

3.5 Webhook Receives Payment Confirmation

Test ID: `E2E-005`
Priority: P1 — High
Note: Requires ngrok tunnel

Step	Action	Expected Result
1	Complete payment flow	Payment succeeds on Chapa
2	Chapa sends webhook to `/api/payments/webhook`	Webhook received
3	Webhook processes payment	Order updated
4	User arrives at confirmation page	Already verified (idempotent)

Success Criteria:

- ☑ Webhook endpoint accessible via ngrok
- ☑ Webhook updates order status
- ☑ Duplicate webhooks handled idempotently

3.6 Duplicate Verification Prevention

Test ID: `E2E-006`
Priority: P1 — High

Step	Action	Expected Result
1	Complete successful payment	Order confirmed
2	Manually call `/api/payments/verify?tx_ref=...` again	API called
3	Check response	Returns `success: true, message: "Payment already verified"`
4	Check database	No duplicate updates, timestamps unchanged

Success Criteria:

- ☑ Idempotent response returned
- ☑ No duplicate status_history entries
- ☑ `payment_completed_at` not overwritten

3.7 Unauthenticated Checkout Attempt

Test ID: `E2E-007`
Priority: P1 — High

Step	Action	Expected Result
1	Log out of application	Session cleared
2	Attempt to create order via API	POST to `/api/orders`
3	Check response	Returns `401 Unauthorized`

Success Criteria:

- ☑ Order creation blocked for unauthenticated users
- ☑ No order record created in database

3.8 Invalid Transaction Reference

Test ID: `E2E-008`
Priority: P2 — Medium

Step	Action	Expected Result
1	Navigate to `/order-confirmation?tx_ref=INVALID_REF`	Page loads
2	Verification runs with invalid tx_ref	API called
3	Check response	Returns `404 Order not found`
4	UI displays error	Appropriate error message shown

Success Criteria:

- ☑ Graceful handling of invalid tx_ref
- ☑ No application crash
- ☑ User-friendly error message

---

4. Database Verification Checklist

After each test, verify the following in Supabase:

4.1 Orders Table

```sql
SELECT
  id,
  order_number,
  status,
  payment_status,
  payment_provider,
  tx_ref,
  payment_completed_at,
  total_amount,
  created_at
FROM orders
ORDER BY created_at DESC
LIMIT 5;
```

Expected for successful payment:

- `status`: `order_confirmed`
- `payment_status`: `paid`
- `payment_provider`: `chapa`
- `tx_ref`: `POL-{uuid}-{timestamp}`
- `payment_completed_at`: Recent timestamp

4.2 Status History Verification

```sql
SELECT
  order_number,
  status_history
FROM orders
WHERE tx_ref = 'YOUR_TX_REF';
```

Expected entries:

1. `pending` — "Order initiated, awaiting payment"
2. `order_confirmed` — "Payment verified via Chapa" OR "Payment confirmed via Webhook"

---

5. API Response Validation

5.1 POST /api/orders (Success)

```json
{
  "success": true,
  "order": {
    "id": "uuid",
    "order_number": "POL-XXXXX",
    "status": "pending",
    "payment_status": "pending_payment"
  },
```

```
  "checkout_url": "https://checkout.chapa.co/..."
}
```

5.2 GET /api/payments/verify (Success)

```
{
  "success": true,
  "message": "Payment verified successfully",
  "order": {
    "id": "uuid",
    "status": "order_confirmed",
    "payment_status": "paid"
  }
}
```

5.3 GET /api/payments/verify (Already Verified)

```
{
  "success": true,
  "message": "Payment already verified",
  "order": {
    "id": "uuid",
    "status": "order_confirmed",
    "payment_status": "paid"
  }
}
```

5.4 GET /api/payments/verify (Failed)

```
{
  "success": false,
  "message": "Payment verification failed",
  "details": "failed"
}
```

---

6. Browser DevTools Verification

6.1 Network Tab Checks

Request	Method	Expected
`/api/orders`	POST	201, returns `checkout_url`
Redirect to Chapa	GET	200, Chapa hosted page loads

Request	Method	Expected
Return from Chapa	GET	Redirect to `/order-confirmation?tx_ref=...`
`/api/payments/verify?tx_ref=...`	GET	200, verification response
`/api/send-order-email`	POST	200, email triggered

6.2 Security Checks

- ☐ No `CHAPA_SECRET_KEY` visible in any client-side code
- ☐ No card input fields on our domain
- ☐ All API calls use HTTPS in production
- ☐ Proper CORS headers on API routes

---

7. Test Execution Log

Test ID	Scenario	Status	Notes	Tester	Date
E2E-001	Happy Path	☐ Pending			
E2E-002	Failed Payment	☐ Pending			
E2E-003	User Cancel	☐ Pending			
E2E-004	Network Failure	☐ Pending			
E2E-005	Webhook	☐ Pending			
E2E-006	Duplicate Verify	☐ Pending			
E2E-007	Unauthenticated	☐ Pending			
E2E-008	Invalid tx_ref	☐ Pending			

---

8. Known Limitations & Future Enhancements

Current Limitations

1. **No retry payment mechanism** — If payment fails, user must create new order
2. **No payment status polling** — Relies on user returning to confirmation page
3. **No webhook signature verification** — Webhook accepts all requests (add Chapa signature validation for production)

Recommended Enhancements

1. Add webhook signature verification using Chapa's webhook secret
2. Implement payment retry without creating new order
3. Add admin notification for failed payments
4. Implement payment status reconciliation job (cron)
5. Add payment analytics and reporting

---

9. Production Readiness Checklist

Before deploying to production:

- ☐ Replace test `CHAPA_SECRET_KEY` with production key
- ☐ Update `NEXT_PUBLIC_APP_URL` to `https://printonline.et`
- ☐ Configure webhook URL in Chapa dashboard: `https://printonline.et/api/payments/webhook`
- ☐ Enable HTTPS enforcement
- ☐ Add webhook signature verification
- ☐ Set up monitoring/alerts for payment failures
- ☐ Configure rate limiting on API routes
- ☐ Test with real (small amount) transaction
- ☐ Document rollback procedure

Document Version: 1.0
Last Updated: 2026-03-17
Next Review: After E2E testing completion